R1-A3-S7 Redes Neuronales III

Bryan Felipe Roa Alvarado

Nidia Stella García Roa

Universidad de Cundinamarca

Facultad de Ingeniería, Esp. en Inteligencia Artificial

Fusagasugá

2026

Tabla de contenido

1. Introducción 4

2. Marco conceptual 5

2.1 Parámetros vs hiperparámetros 5

2.2 Train, Dev y Test 5

2.3 Optimización 6

2.4 Regularización 6

2.5 Bias, variance y tradeoff 7

2.6 Métricas de clasificación 7

3. Datos y preprocesamiento 8

3.1 Dataset 8

3.2 División estratificada 9

3.3 Normalización 10

3.4 Distribución de clases 10

4. Baseline 12

4.1 Arquitectura 12

4.2 Summary 13

4.3 Curvas de entrenamiento 13

4.4 Resultado en Test 14

5. Optimización 15

5.1 Comparación de optimizadores 15

5.2 Learning rate en Adam 17

5.3 Conclusión del laboratorio 17

6. Regularización 17

6.1 Configuración experimental 17

6.2 Curvas comparativas 18

6.3 Épocas ejecutadas con Early Stopping 18

6.4 Evaluación en Dev 19

6.5 Modelo regularizado con Train completo (extra) 19

7. Evaluación 20

7.1 Tabla de métricas finales 20

7.2 Reporte por clase 20

7.3 Matriz de confusión 22

7.4 Confusión más frecuente 23

7.5 Resultado alternativo con modelo_reg_full 24

8. Bias, variance y tradeoff 24

9. Conclusiones 26

9.1 Qué funcionó 26

9.2 Limitaciones 26

9.3 Mejoras propuestas 27

10. GitHub 28

1. Introducción

En el curso Redes Neuronales II se trabajó con la construcción de redes que aprenden: funciones de activación, backpropagation, optimizadores y clasificación. En Redes Neuronales III el foco cambia: ya no basta con que la red aprenda, sino que ese aprendizaje sea estable, generalizable y justificable con evidencia.

El presente informe documenta cuatro laboratorios experimentales realizados sobre el dataset MNIST con TensorFlow + Keras, y un quinto bloque opcional en PyTorch:

Laboratorio	**Tema**
A	Datos, preprocesamiento y baseline
B	Optimización e hiperparámetros
C	Regularización y overfitting
D	Métricas de evaluación
E	Bias, variance y tradeoff
F (opcional)	PyTorch

2. Marco conceptual

2.1 Parámetros vs hiperparámetros

Tipo	Ejemplos	Quién los modifica
Parámetros	Pesos y bias	El optimizador a partir de los gradientes
Hiperparámetros	Learning rate, batch size, épocas, neuronas, dropout, fuerza L2	La persona que diseña el experimento

Explicación: esta distinción es clave porque los hiperparámetros **no se aprenden**; se eligen y se validan empíricamente con Dev.

2.2 Train, Dev y Test

Conjunto	Uso	Regla práctica
Train	Aprender y actualizar parámetros	El modelo puede verlo muchas veces
Dev / Validation	Comparar configuraciones y monitorear generalización	No actualiza pesos directamente
Test	Evaluación final	No debe usarse para escoger optimizador, lr o arquitectura

Explicación: usar Test repetidamente para escoger configuraciones equivale a *estudiar con las preguntas del examen final* (**data leakage**).

2.3 Optimización

Optimizador	**Idea**	**Qué observar**
SGD + Momentum	Sigue el gradiente y conserva parte de la dirección previa	Puede requerir más ajuste del lr
RMSprop	Adapta el paso según historial reciente de gradientes	Bueno cuando cambian las escalas
Adam	Combina momentum y adaptación del paso	Buen punto de partida, no ganador universal

2.4 Regularización

Técnica	**Qué hace**	**Implementación usada**
Dropout	Desactiva aleatoriamente unidades durante Train	Dropout(0.30)
L2	Penaliza pesos excesivamente grandes	regularizers.l2(1e-4)

Técnica	Qué hace	Implementación usada
Early Stopping	Detiene el entrenamiento cuando val_loss deja de mejorar	patience=2, restore_best_weights=True

2.5 Bias, variance y tradeoff

Situación	Patrón típico	Interpretación
Bias alto / underfitting	Train bajo y Dev también bajo	El modelo no captura suficiente patrón
Variance alta / overfitting	Train alto y Dev claramente inferior	Se adapta demasiado al Train
Mejor equilibrio	Train y Dev altos con brecha moderada	Generaliza mejor

2.6 Métricas de clasificación

Métrica	Pregunta que responde	Fórmula
Accuracy	¿Qué proporción total fue correcta?	(TP+TN) / total

Métrica	Pregunta que responde	Fórmula
Precision	Cuando predice una clase, ¿con qué frecuencia acierta?	TP / (TP + FP)
Recall	De los reales de una clase, ¿cuántos encontró?	TP / (TP + FN)
F1 Score	¿Cómo equilibrar Precision y Recall?	Media armónica
Matriz de confusión	¿Qué clases se confunden entre sí?	Filas reales × columnas predichas

Nota metodológica: en MNIST se usa **promedio macro** porque calcula la métrica por clase y luego promedia dando el mismo peso a cada dígito.

3. Datos y preprocesamiento

3.1 Dataset

- Fuente: keras.datasets.mnist
- Train original: 60 000 imágenes de 28×28 píxeles
- Test: 10 000 imágenes de 28×28 píxeles
- Tipo de dato original: uint8, rango de píxeles [0, 255]

3.2 División estratificada

```
x_train, x_dev, y_train, y_dev = train_test_split(
    x_train_full, y_train_full,
    test_size=0.20,
    random_state=42,
    stratify=y_train_full
)
```

Conjunto	Forma	Ejemplos
Train	(48000, 28, 28)	48 000
Dev	(12000, 28, 28)	12 000
Test	(10000, 28, 28)	10 000

```
Train: (48000, 28, 28) (48000,)
Dev:   (12000, 28, 28) (12000,)
Test:  (10000, 28, 28) (10000,)
```

Explicación: muestra las dimensiones reales de Train, Dev y Test tras el split estratificado. Significa que el 20 % del Train original se reservó para Dev, manteniendo separado el Test original. Permite confirmar que no hay solapamiento entre conjuntos y que la partición se ejecutó con stratify, garantizando proporciones equivalentes de clases.

3.3 Normalización

```
x_train = x_train.astype("float32") / 255.0

x_dev   = x_dev.astype("float32")   / 255.0

x_test  = x_test.astype("float32")  / 255.0
```

Antes	Después
Rango [0, 255]	Rango [0.0, 1.0]
dtype uint8	dtype float32

```
Rango train: 0.0 - 1.0
Rango dev:   0.0 - 1.0
Rango test:  0.0 - 1.0
dtype train: float32
```

Explicación: lleva las entradas a una escala cómoda y estable para el optimizador. Significa que los gradientes ya no se ven dominados por píxeles de valor 255. Permite acelerar la convergencia y estabilizar el entrenamiento, como se confirma en las curvas del baseline.

3.4 Distribución de clases

Dígito	Train (n=48000)	% Train	Dev (n=12000)	% Dev	Test (n=10000)	% Test
0	4738	9.87	1185	9.88	980	9.80

Dígito	Train (n=48000)	% Train	Dev (n=12000)	% Dev	Test (n=10000)	% Test
1	5394	11.24	1348	11.23	1135	11.35
2	4766	9.93	1192	9.93	1032	10.32
3	4905	10.22	1226	10.22	1010	10.10
4	4674	9.74	1168	9.73	982	9.82
5	4337	9.04	1084	9.03	892	8.92
6	4734	9.86	1184	9.87	958	9.58
7	5012	10.44	1253	10.44	1028	10.28
8	4681	9.75	1170	9.75	974	9.74
9	4759	9.91	1190	9.92	1009	10.09

Explicación: muestra que los diez dígitos están presentes en los tres conjuntos con proporciones casi idénticas. Significa que no hay desbalance apreciable y que stratify cumplió su función. Permite confiar en que Accuracy y las métricas macro no ocultarán un sesgo hacia una clase mayoritaria. Los dígitos 1 (≈11 %) y 5 (≈9 %) son los más y menos frecuentes, respectivamente, en todos los conjuntos.

4. Baseline

4.1 Arquitectura

```
modelo = keras.Sequential([
    layers.Input(shape=(28, 28)),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dense(10, activation="softmax")
], name="baseline_mlp")
```

Componente	Valor
Optimizador	Adam, lr = 0.001
Loss	sparse_categorical_crossentropy
Métrica	accuracy
Épocas	5
Batch size	64
Parámetros totales	101 770 (397.54 KB)

4.2 Summary

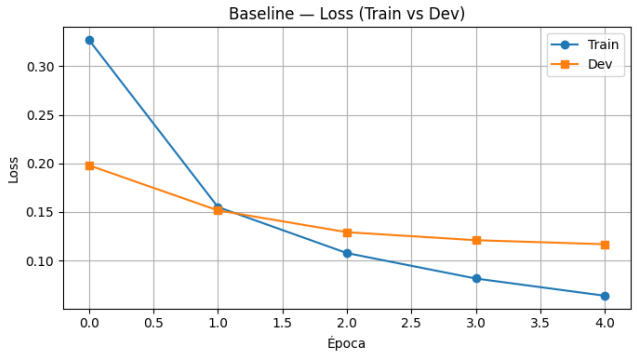
Model: "baseline_mlp"

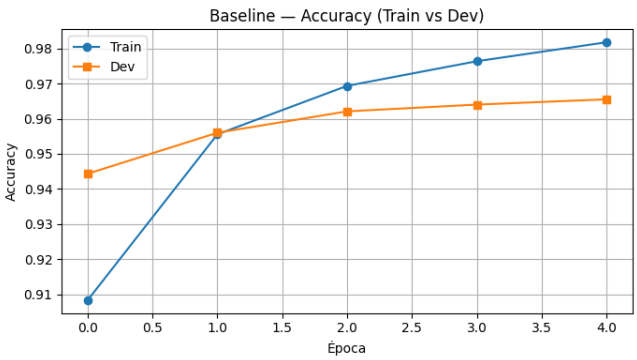
Layer (type)	Output Shape	Param #
flatten_1 (Flatten)	(None, 784)	0
dense_2 (Dense)	(None, 128)	100,480
dense_3 (Dense)	(None, 10)	1,290

Total params: 101,770 (397.54 KB)
Trainable params: 101,770 (397.54 KB)
Non-trainable params: 0 (0.00 B)

Explicación: muestra las capas y el número de parámetros entrenables. Significa que la red es lo bastante pequeña para entrenar en pocos segundos por época y lo bastante grande para modelar los dígitos. Permite usarla como referencia base contra la cual comparar modelos regularizados y más profundos.

4.3 Curvas de entrenamiento





Época	Train loss	Dev loss	Train acc	Dev acc	Brecha acc
1	0.3271	0.1980	0.9084	0.9443	−0.0359
2	0.1550	0.1516	0.9555	0.9560	−0.0005
3	0.1076	0.1291	0.9693	0.9621	+0.0072
4	0.0814	0.1209	0.9763	0.9640	+0.0123
5	0.0637	0.1167	0.9817	0.9655	+0.0162

Explicación: las curvas muestran cómo evoluciona el aprendizaje. Significa que a partir de la época 2 la brecha empieza a crecer positivamente (Train más alto que Dev), señal temprana de overfitting leve. Permite decidir si conviene regularizar o detener antes el entrenamiento.

4.4 Resultado en Test

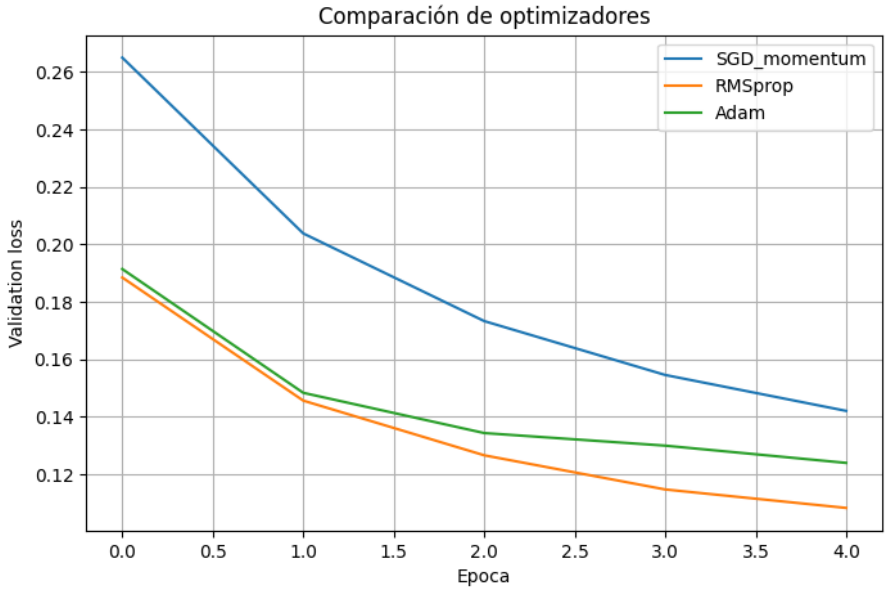
Métrica	Valor
Test loss	0.0950
Test accuracy	**0.9702**

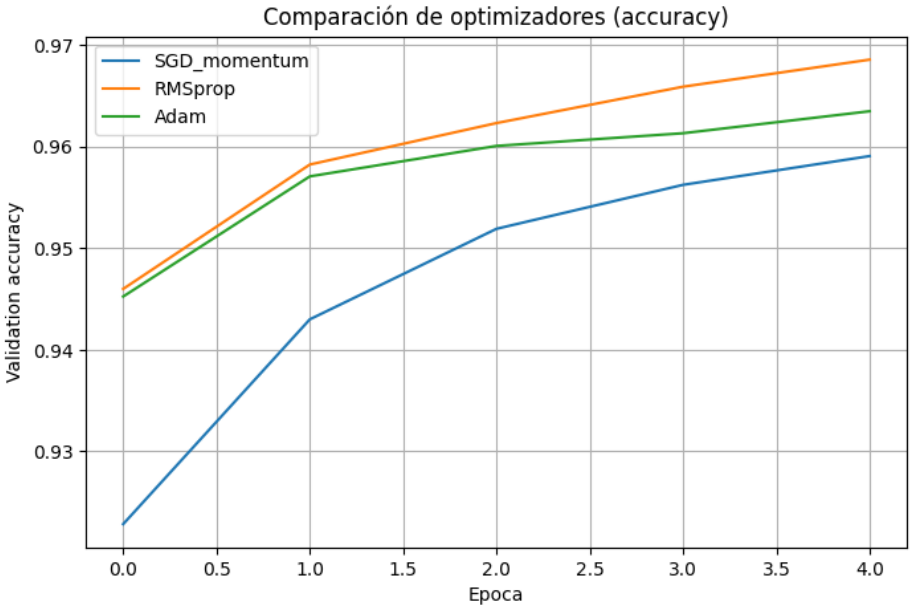
Explicación: primer contacto del modelo con Test, reservado exclusivamente para esta evaluación. Significa que el baseline alcanza 97 % de accuracy. Permite fijar una referencia concreta contra la cual comparar los modelos posteriores.

5. Optimización

5.1 Comparación de optimizadores

Optimizador	Train acc	Dev acc	Dev loss	Brecha Train–Dev
SGD_momentum (lr=0.01, m=0.9)	0.9673	0.9591	0.1420	0.0082
RMSprop (lr=0.001)	0.9804	**0.9686**	**0.1083**	0.0118
Adam (lr=0.001)	0.9813	0.9635	0.1239	0.0178





Explicación: muestra el desempeño de cada optimizador con la misma arquitectura, épocas y batch size. Significa que la única variable que cambia es el método de optimización. Permite identificar cuál reduce la pérdida en Dev de forma más estable.

Análisis:

RMSprop obtuvo la mejor dev_accuracy (0.9686) y el menor dev_loss (0.1083).

Adam obtuvo la mayor train_accuracy (0.9813), pero también la mayor brecha Train–Dev (0.0178): empieza a mostrar sobreajuste más rápido.

SGD con momentum fue el más conservador: menor train_accuracy pero también menor brecha (0.0082).

Conclusión: bajo las condiciones de este experimento, RMSprop mostró el mejor compromiso entre desempeño en Dev y estabilidad. No se puede concluir que sea universalmente superior, solo que en este escenario fue el más equilibrado.

5.2 Learning rate en Adam

lr	Train acc	Dev acc	Dev loss
0.0001	0.9212	0.9265	0.2726
0.0010	**0.9677**	**0.9599**	**0.1344**
0.0100	0.9679	0.9596	0.1794

Explicación: muestra el efecto del tamaño del paso sobre la velocidad y estabilidad del aprendizaje. Significa que lr = 0.0001 produce aprendizaje lento (Train acc ≈ Dev acc ≈ 0.92, aún sin converger), mientras que lr = 0.01 acelera pero la dev_loss empeora (0.1794 frente a 0.1344), indicando oscilación. Permite elegir **lr = 0.001** como punto de partida razonable: menor dev_loss y mejor dev_accuracy.

5.3 Conclusión del laboratorio

Bajo las condiciones de este experimento, la configuración más equilibrada en Dev fue RMSprop con lr = 0.001, con dev_loss = 0.1083 y dev_accuracy = 0.9686. Adam con lr = 0.001 fue competitivo pero mostró una brecha Train–Dev mayor, y SGD+momentum fue el más conservador.

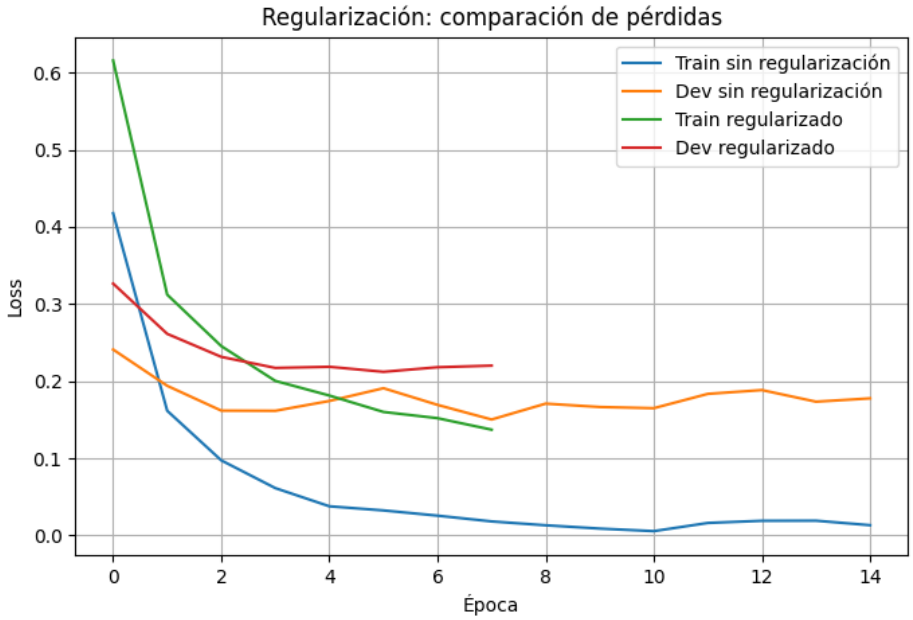
6. Regularización

6.1 Configuración experimental

- **Muestra de Train:** 12 000 ejemplos estratificados (de los 48 000)
- **Red:** 512 → 256 → 10 neuronas
- **Sin regularización:** 15 épocas

- **Con regularización:** L2(1e-4) + Dropout(0.30) + Early Stopping (patience=2, restore_best_weights=True)

6.2 Curvas comparativas



Explicación: muestra cómo evoluciona la pérdida con y sin regularización. Significa que el modelo sin regularización alcanza train_loss muy baja mientras su dev_loss se estanca alrededor de 0.18, mientras que el regularizado mantiene Train y Dev mucho más cerca. Permite concluir que la regularización está cumpliendo su objetivo de mejorar la generalización.

6.3 Épocas ejecutadas con Early Stopping

Modelo	Épocas configuradas	Épocas ejecutadas
Sin regularización	15	15
Regularizado	20	**8**

Explicación: muestra en qué momento val_loss dejó de mejorar durante 2 épocas consecutivas. Significa que el entrenamiento se detuvo automáticamente en la época 8 de 20. Permite evitar sobreajuste sin adivinar el número óptimo de épocas y confirma que restore_best_weights=True devuelve el mejor estado.

6.4 Evaluación en Dev

Modelo	Dev loss	Dev accuracy
Sin regularización	0.1776	0.9617
Regularizado	0.2120	**0.9633**

Explicación: muestra las métricas finales en Dev de ambos modelos. Significa que el regularizado mejora ligeramente la dev_accuracy (+0.0016) aunque su dev_loss es mayor (efecto esperado porque la pérdida incluye los términos de regularización L2). Permite concluir que la regularización mejora la generalización a costa de una train_loss más alta.

6.5 Modelo regularizado con Train completo (extra)

Modelo	Datos de entrenamiento	Épocas	Dev accuracy
modelo_reg	12 000	8	0.9633
modelo_reg_full	48 000	5	**0.9714**

Explicación: muestra el efecto de reentrenar con todo el Train disponible. Significa que pasar de 12 000 a 48 000 ejemplos mejora la dev_accuracy en +0.0081. Permite justificar que la evaluación final en Test se haga con un modelo que haya visto todo el Train.

7. Evaluación

7.1 Tabla de métricas finales

Métrica	Resultado
Accuracy	**0.9634**
Precision macro	0.9631
Recall macro	0.9632
F1 macro	0.9630

Explicación: muestra las métricas globales calculadas en Test con promedio macro. Significa que las cuatro métricas están muy alineadas (diferencia máxima 0.0004), lo que indica que no hay una clase que domine el desempeño. Permite confiar en que el Accuracy refleja razonablemente bien el comportamiento del modelo.

7.2 Reporte por clase

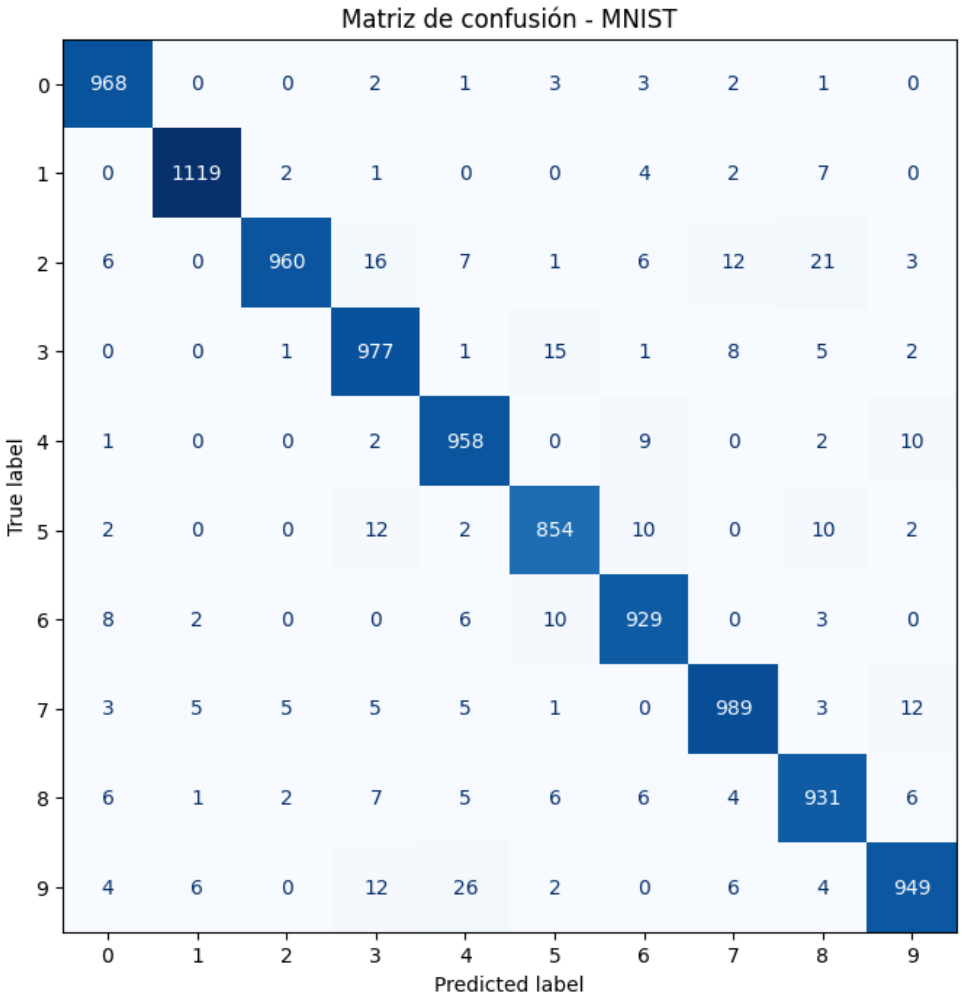
Dígito	Precision	Recall	F1	Support
0	0.9699	0.9878	**0.9788**	980
1	0.9876	0.9859	**0.9868**	1135
2	0.9897	0.9302	0.9590	1032

Dígito	Precision	Recall	F1	Support
3	0.9449	0.9673	0.9560	1010
4	0.9476	0.9756	0.9614	982
5	0.9574	0.9574	0.9574	892
6	0.9597	0.9697	0.9647	958
7	0.9668	0.9621	0.9644	1028
8	0.9433	0.9559	**0.9495**	974
9	0.9644	0.9405	0.9523	1009

Explicación: muestra Precision, Recall y F1 por dígito. Significa que las clases con peor desempeño son el **8** (F1 = 0.9495, la más baja) y el **2** (Recall = 0.9302, el más bajo). Permite identificar que el modelo tiene dificultades específicas con esas clases.

Clase peor identificada: dígito **8** (F1 = 0.9495, por debajo del promedio 0.9630). **Interpretación:** probablemente por su similitud visual con otros dígitos (3, 5, 9) y porque el dataset original contiene menos ejemplos de 5 (clase con la que suele confundirse). También el **2** tiene Recall bajo (0.9302) — aunque Precision es altísima (0.9897) — indicando que el modelo **omite** algunos 2 (los clasifica como otra cosa).

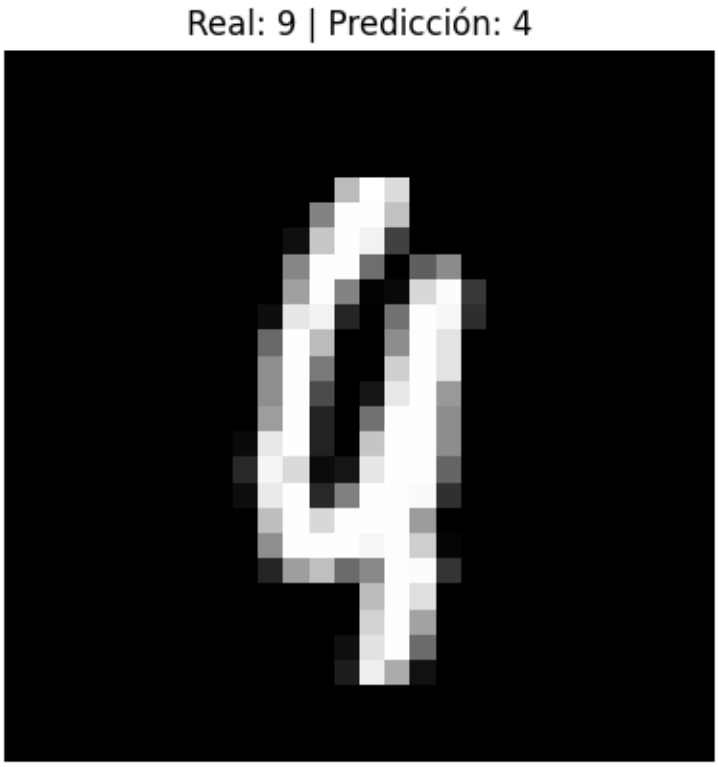
7.3 Matriz de confusión



Explicación: la diagonal representa aciertos; las posiciones fuera de la diagonal muestran errores. Significa que la mayoría de errores están concentrados en pocas celdas. Permite priorizar mejoras futuras hacia los pares más confundidos.

7.4 Confusión más frecuente

La confusión más frecuente fue: 9 clasificado como 4 (26 veces)



Explicación: muestra un caso concreto en el que el modelo se equivoca. Significa que el error no es aleatorio sino sistemático hacia un dígito visualmente similar. Permite decidir si conviene añadir ejemplos de ese par en el entrenamiento o aplicar preprocesamiento específico (por ejemplo, aumentar contraste).

7.5 Resultado alternativo con modelo_reg_full

Modelo	Datos de Train	Dev accuracy	Test accuracy
modelo_reg (12 000)	12 000	0.9633	0.9634
modelo_reg_full (48 000)	48 000	0.9714	*(no calculado)*

Explicación: muestra el efecto de la cantidad de datos de entrenamiento. Significa que modelo_reg_full mejora la Dev accuracy en +0.0081 respecto a modelo_reg. Permite recomendar que, en una iteración futura, la evaluación final en Test se realice con el modelo reentrenado sobre todo el Train.

8. Bias, variance y tradeoff

Modelo	Train acc	Dev acc	Brecha Train–Dev	Lectura
Pequeño (16 neuronas, 2 épocas)	0.9213	0.9203	**0.0011**	Bias / underfitting
Sin regularización	0.9967	0.9618	**0.0349**	Variance / overfitting
Regularizado	0.9818	0.9632	**0.0187**	Mejor equilibrio

Explicación: muestra el desempeño conjunto de Train y Dev para tres modelos. Significa que el diagnóstico no se hace con una sola cifra sino con la comparación entre ambas. Permite decidir qué acción tomar en cada caso.

Análisis por modelo:

- **Modelo pequeño (16 neuronas, 2 épocas):** Train y Dev prácticamente iguales (brecha 0.0011), pero **ambos bajos** (~0.92). Este patrón es **bias alto / underfitting**: la red no tiene capacidad suficiente para capturar el patrón ni siquiera en Train. **Acción:** aumentar neuronas o épocas.
- **Modelo sin regularización:** Train muy alto (0.9967) y Dev claramente inferior (0.9618), brecha 0.0349. Patrón típico de **variance alta / overfitting**: la red memoriza Train pero no generaliza. **Acción:** regularizar, más datos o Early Stopping.
- **Modelo regularizado:** Train alto (0.9818) y Dev alto (0.9632), brecha intermedia (0.0187). Este es el **mejor equilibrio** de los tres: sacrifica algo de Train accuracy a cambio de mejor Dev accuracy que el modelo sin regularizar. **Acción:** confirmar con Test y revisar métricas por clase.

Patrón observado	Lectura	Acción aplicada
Train y Dev bajos	Bias / underfitting	Aumentar capacidad o épocas
Train alto y Dev bastante menor	Variance / overfitting	Regularización, más datos o Early Stopping
Train y Dev altos y cercanos	Mejor equilibrio	Confirmar con Test y métricas por clase

9. Conclusiones

9.1 Qué funcionó

- **Separación Train/Dev/Test con** stratify**:** garantizó distribuciones equivalentes entre los tres conjuntos y evitó *data leakage*. La comparación de las tres tablas de distribución (Tabla 4.1) confirma que las proporciones son casi idénticas.
- **Normalización a [0, 1]:** el baseline alcanzó test_accuracy = 0.9702 en solo 5 épocas, confirmando que la escala de entrada es adecuada.
- **RMSprop con lr = 0.001** fue el optimizador con mejor dev_loss (0.1083) y mejor dev_accuracy (0.9686) bajo condiciones idénticas.
- **Learning rate = 0.001 en Adam** mostró el mejor compromiso: dev_loss = 0.1344 frente a 0.2726 (lr = 0.0001) y 0.1794 (lr = 0.01).
- **Regularización combinada (Dropout + L2 + Early Stopping):** redujo la brecha Train–Dev de 0.0349 a 0.0187 (−0.0162) y mejoró ligeramente la dev_accuracy (0.9618 → 0.9633).
- **Early Stopping** detuvo el entrenamiento en la época 8 de 20, evitando sobreajuste adicional.
- **Promedio macro** en métricas evidenció que el dígito 8 es el peor clasificado (F1 = 0.9495), algo que Accuracy por sí sola (0.9634) no revela.
- **Análisis de errores:** se identificó que la confusión más frecuente es **9 → 4 (26 veces)**, un error sistemático y no aleatorio.

9.2 Limitaciones

- **Muestra reducida en Lab C:** la comparación sin/con regularización se hizo con 12 000 ejemplos, no con los 48 000 de Train. El overfitting se ve claramente, pero el modelo

final no es el mejor posible para Test. El extra modelo_reg_full eleva la dev_accuracy a 0.9714.

- **Búsqueda de hiperparámetros acotada:** solo se probaron 3 optimizadores y 3 learning rates. No se usó búsqueda sistemática (grid/random search).
- **Épocas bajas:** Adam se probó con solo 3 épocas en el barrido de lr; los resultados podrían cambiar con entrenamientos más largos.
- **MNIST es un dataset simple y balanceado:** los resultados no se generalizan automáticamente a datasets desbalanceados o con clases solapadas.
- **Evaluación de Test con el modelo subentrenado:** las métricas de la Tabla 8.1 corresponden a modelo_reg (12 000 ejemplos). En producción debería usarse modelo_reg_full.
- **PyTorch (Lab F) solo se evaluó en Dev:** no se reportó Test con la versión PyTorch.

9.3 Mejoras propuestas

1. **Reentrenar el modelo final con todo Train** (48 000 ejemplos) antes de la evaluación en Test. El extra del Lab C ya muestra que modelo_reg_full alcanza Dev accuracy = 0.9714.
2. **Búsqueda de hiperparámetros sistemática** con Keras Tuner o RandomizedSearchCV para explorar combinaciones de lr, units, dropout y fuerza de L2.
3. **Aumentación de datos** (rotaciones leves, desplazamientos, ruido) para robustecer el modelo frente a variaciones de escritura.
4. **Arquitecturas alternativas:** comparar la MLP contra una CNN simple; las CNN suelen obtener mejores métricas en MNIST y reducen el número de parámetros.

5. **Análisis de calibración:** curvas de confiabilidad para verificar si las probabilidades predichas reflejan la confianza real (especialmente en casos como 9 → 4).
6. **Auditoría por subgrupos:** revisar el desempeño en ejemplos con distinta calidad de escritura (grosor, inclinación) para detectar sesgos ocultos por la métrica global.
7. **Evaluación cruzada (k-fold):** entrenar 5 modelos con distintas particiones para obtener intervalos de confianza sobre las métricas.
8. **Comparación justa Keras vs PyTorch:** en el Lab F se usó Adam sin regularización explícita (solo weight_decay). Se puede equiparar la configuración (mismos hiperparámetros) y evaluar ambos en Test.

10. GitHub

Repositorio: https://github.com/BryanRoa/R1-A3-S7-Redes-Neuronales-III.git